

PYSPARK RDD-BASED DATA PROCESSING PIPELINE.

◇ 1. Problem Context (What are we solving?)

You have orders data with schema:

(order_id, date, customer_id, order_status)

Sample:

1,2013-07-25,11599,CLOSED

2,2013-07-25,256,PENDING_PAYMENT

...

Business questions:

1. Count orders per status
 2. Top 10 customers (most orders)
 3. Distinct customers
 4. Customers with most CLOSED orders
-

◇ 2. Spark Session Creation (Execution Entry Point)

```
spark = SparkSession.builder \  
    .config("spark.ui.port", "0") \  
    .config("spark.sql.warehouse.dir", f"/user/{username}/warehouse") \  
    .enableHiveSupport() \  
    .master("yarn") \  
    .getOrCreate()
```

🧠 What's happening internally?

- SparkSession = Unified entry point (replaces SparkContext + SQLContext)
- .master("yarn") → Jobs run on cluster (not locally)
- Driver program starts → connects to cluster manager (YARN)
- Executors are launched on worker nodes

Architecture:

Driver Program



Cluster Manager (YARN)



Executors (distributed across nodes)

◇ 3. Loading Data into RDD

```
orders_rdd = spark.sparkContext.textFile("../orders/*")
```

🧠 Internals:

- File is split into partitions (based on HDFS blocks)
- Each partition is processed by one executor task
- Data is not loaded fully → lazy reference

👉 This is lazy evaluation: no computation happens yet.

◇ 4. Key Transformation Pattern (Map → Reduce)

This pattern appears everywhere in your examples.

◇ 5. Use Case 1: Orders per Category

```
mapped_rdd = orders_rdd.map(lambda x: (x.split(",")[3], 1))
```

```
reduced_rdd = mapped_rdd.reduceByKey(lambda x, y: x + y)
```

```
sorted_rdd = reduced_rdd.sortBy(lambda x: x[1], False)
```

🧠 Step-by-step internal flow:

Step 1: map

"CLOSED" → ("CLOSED", 1)

- Narrow transformation
 - No shuffle
 - Runs independently per partition
-

Step 2: reduceByKey

- Groups by key (order_status)

- Aggregates counts

⚠ Important internal behavior:

💧 Shuffle happens here

- Data is redistributed across nodes based on key
- Same keys go to same executor

Optimization:

- Uses combiner (map-side aggregation)
→ reduces network traffic
-

Step 3: sortBy

- Global sorting → another shuffle
 - Expensive operation
-

◇ 6. Use Case 2: Premium Customers

```
customers_mapped = orders_rdd.map(lambda x: (x.split(",")[2], 1))
```

```
customers_aggregated = customers_mapped.reduceByKey(lambda x, y: x + y)
```

```
top10 = customers_aggregated.sortBy(lambda x: x[1], False).take(10)
```

💡 Insight:

- You're counting orders per customer
- take(10) triggers execution (ACTION)

⚠ Internals:

- take(10) is optimized:
 - Doesn't sort entire dataset if possible
 - Uses partial aggregation + top-k logic
-

◇ 7. Use Case 3: Distinct Customers

```
distinct_customers = orders_rdd.map(lambda x: x.split(",")[2]).distinct()
```

```
count = distinct_customers.count()
```

💡 Internals:

- `distinct()` → implemented as:
 - `map` → `(customer_id, null)`
 - `reduceByKey`
 - Causes shuffle
-

◇ 8. Use Case 4: Customers with CLOSED Orders

```
filtered = orders_rdd.filter(lambda x: x.split(",")[3] == "CLOSED")
```

```
mapped = filtered.map(lambda x: (x.split(",")[2], 1))
```

```
aggregated = mapped.reduceByKey(lambda x, y: x + y)
```

```
sorted = aggregated.sortBy(lambda x: x[1], False)
```

🧠 Pipeline breakdown:

1. `filter` → narrow transformation (no shuffle)
 2. `map` → narrow
 3. `reduceByKey` → shuffle
 4. `sortBy` → shuffle
-

◇ 9. Transformation vs Action

Transformations (lazy):

- `map`
- `filter`
- `reduceByKey`
- `sortBy`
- `distinct`

Actions (trigger execution):

- `collect()`
 - `take()`
 - `count()`
-

◇ 10. DAG (Directed Acyclic Graph)

Spark builds a DAG like:

textFile



map



filter



reduceByKey ← shuffle boundary (stage split)



sortBy ← another stage



action

🧠 Key idea:

- Spark optimizes execution plan before running
- Splits into stages
- Each stage = pipeline of narrow transformations

◇ 11. Important Concepts Explained

◇ map vs reduce vs reduceByKey

Operation	Output	Shuffle	Use Case
map	same size	✗	transform rows
reduce	1 value	✗	global aggregation
reduceByKey per key		☑	grouped aggregation

◇ filter

- Keeps subset of data
 - No shuffle
-

◇ distinct

- Removes duplicates
 - Requires shuffle
-

◇ sortBy

- Expensive global operation
 - Always causes shuffle
-

◇ 12. Performance Insights (Senior-Level)

🔦 Bottlenecks in this pipeline:

1. Shuffle-heavy operations

- reduceByKey
- sortBy
- distinct

👉 These involve:

- Disk I/O
 - Network transfer
 - Serialization
-

🚀 Optimizations:

☑ Use reduceByKey over groupByKey

- Less memory
 - Uses combiner
-

☑ Avoid full sortBy

Instead:

`takeOrdered(10, key=lambda x: -x[1])`

👉 avoids full shuffle

☒ Use DataFrames instead of RDD

Why?

- Catalyst optimizer
 - Tungsten execution engine
 - Better memory management
-

☒ Partitioning strategy

- Control with:

`rdd.partitionBy(n)`

☒ Cache intermediate results

`rdd.cache()`

◇ 13. What This Example Teaches

This is a classic MapReduce pattern implemented in Spark RDDs:

Raw Data → Map → Shuffle → Reduce → Output

◇ 14. Real-World Perspective

In production:

✗ You rarely use RDDs directly

☒ You use:

- DataFrames
- Spark SQL

Because:

- Automatic optimization
 - Less code
 - Faster execution
-

◇ 15. If I Were Designing This Pipeline

I would rewrite using DataFrames:

```
df.groupBy("order_status").count()
```

Instead of manual RDD transformations.

◇ Final Takeaway

This example demonstrates:

- Core Spark execution model
- Lazy evaluation
- DAG + stages
- Shuffle mechanics
- MapReduce paradigm

But it's low-level Spark.

☞ Think of this as:

“Understanding assembly before writing high-level code”

CLASSIC PYSPARK RDD-BASED ANALYTICS PIPELINE

◆ 1. Problem Context (What you're solving)

You have **orders data** with schema:

(order_id, date, customer_id, order_status)

Example:

1,2013-07-25,11599,CLOSED

2,2013-07-25,256,PENDING_PAYMENT

...

Business Questions:

1. Count orders per status
2. Top 10 customers (most orders)
3. Count distinct customers
4. Customers with most **CLOSED** orders

◆ 2. Spark Session (Entry Point)

```
from pyspark.sql import SparkSession
import getpass
```

```
username = getpass.getuser()
```

```
spark = SparkSession \
    .builder \
    .config("spark.ui.port", "0") \
    .config("spark.sql.warehouse.dir", f"/user/{username}/warehouse") \
    .enableHiveSupport() \
    .master("yarn") \
    .getOrCreate()
```

Key Concepts:

Component	Why it matters
SparkSession	Entry point to Spark (replaces SparkContext + SQLContext)
master("yarn")	Runs on cluster (distributed processing)
enableHiveSupport()	Allows querying Hive tables
config(...)	Environment tuning

 Under the hood:
Spark creates a **driver + executors architecture**.

◆ 3. Loading Data into RDD

```
orders_rdd = spark.sparkContext.textFile("/public/trendytech/retail_db/orders/*")
```

💡 **What happens:**

- Reads **distributed files**
- Creates an **RDD (Resilient Distributed Dataset)**
- Each line = 1 record

👉 RDD = low-level distributed collection

◆ 4. Core Transformations & Actions

Before diving into each use case, understand this:

◆ Transformations (lazy)

- `map()`
- `filter()`
- `reduceByKey()`
- `sortBy()`

👉 These **do NOT execute immediately**

◆ Actions (trigger execution)

- `collect()`
- `count()`
- `take()`

👉 Execution happens only when an **action is called**

◆ 5. Use Case 1: Orders per Category

```
mapped_rdd = orders_rdd.map(lambda x: (x.split(",")[3], 1))
```

```
reduced_rdd = mapped_rdd.reduceByKey(lambda x, y: x + y)
```

```
sorted_rdd = reduced_rdd.sortBy(lambda x: x[1], False)
```

💡 **Step-by-step:**

1. `map()`

```
"CLOSED" → ("CLOSED", 1)
```

👉 Converts raw data → key-value pair

2. reduceByKey()

("CLOSED", 1), ("CLOSED", 1) → ("CLOSED", 2)

👉 Distributed aggregation

✓ Combines data **locally before shuffle** → efficient

3. sortBy()

Sort by count descending

⚠ Important Insight:

- reduceByKey is **better than groupByKey**
 - It minimizes data movement (huge in big data)
-

◆ 6. Use Case 2: Premium Customers

```
customers_mapped = orders_rdd.map(lambda x: (x.split(",")[2], 1))
```

```
customers_agg = customers_mapped.reduceByKey(lambda x, y: x + y)
```

```
top10 = customers_agg.sortBy(lambda x: x[1], False).take(10)
```

💡 Logic:

- Extract customer_id
 - Count number of orders per customer
 - Sort descending
 - Take top 10
-

🔥 Real-world meaning:

👉 Identifies **high-value customers**

◆ 7. Use Case 3: Distinct Customers

```
distinct_customers = orders_rdd.map(lambda x: x.split(",")[2]).distinct()
```

```
distinct_customers.count()
```

💡 Key Functions:

distinct()

- Removes duplicates
- Internally does **shuffle + hashing**

count()

- Action → triggers execution

Performance Note:

`distinct()` is expensive (shuffle-heavy)

8. Use Case 4: Customers with Max CLOSED Orders

```
filtered = orders_rdd.filter(lambda x: x.split(",")[3] == "CLOSED")
```

```
mapped = filtered.map(lambda x: (x.split(",")[2], 1))
```

```
agg = mapped.reduceByKey(lambda x, y: x + y)
```

```
sorted = agg.sortBy(lambda x: x[1], False)
```

Pipeline:

1. **filter()**
→ Keep only CLOSED orders
 2. **map()**
→ Convert to (customer_id, 1)
 3. **reduceByKey()**
→ Count CLOSED orders per customer
 4. **sortBy()**
→ Find top customers
-

9. Important Functions Explained Deeply

1. map()

 Transforms each record independently

Input: ["1,2013,...,CLOSED"]

Output: [("CLOSED",1)]

- ✓ Parallelizable
 - ✓ No shuffle
-

◆ 2. filter()

👉 Keeps only matching records

`x.split(",")[3] == "CLOSED"`

- ✓ Narrow transformation (fast)
-

◆ 3. reduceByKey()

👉 Aggregates values per key

`("A",1), ("A",1) → ("A",2)`

⚡ Why it's powerful:

- Combines data **before shuffle**
 - Reduces network I/O
-

◆ 4. sortBy()

👉 Global sorting

⚠ Expensive → requires shuffle

◆ 5. distinct()

👉 Removes duplicates

⚠ Uses shuffle → costly

◆ 6. take(n)

👉 Returns first n elements

- ✓ Efficient for sampling
-

◆ 7. collect()

👉 Brings ALL data to driver

⚠ Dangerous for big data (can crash memory)

◆ 10. Transformation vs Action (Critical Concept)

Type	Example	Execution
Transformation	map, filter	Lazy
Action	count, collect	Triggers job

👉 Spark builds a **DAG (execution plan)** and runs it only on action.

◆ 11. Optimization Insights (Senior-Level)

🚀 1. Avoid collect()

- Use take() or show()
-

🚀 2. Prefer reduceByKey() over groupByKey()

- Less shuffle
-

🚀 3. Use DataFrames instead of RDDs (modern Spark)

RDD is:

- Low-level
 - Not optimized by Catalyst optimizer
-

🚀 4. Cache when reused

orders_rdd.cache()

◆ 12. Big Picture Flow

Raw Data → RDD → Transformations → Shuffle → Aggregation → Action

◆ 13. Final Thought

This entire example is a **textbook MapReduce pattern implemented in Spark**:

Phase Operation

Map Extract key

Phase Operation

Shuffle Group by key

Reduce Aggregate

- ❶ Same logic using DataFrames (industry standard)
 - ❷ DAG (execution plan) visualization / understanding
-

◆ 1. Rewriting Everything Using DataFrames (Production Approach)

🚀 Step 1: Load Data

```
from pyspark.sql import SparkSession
```

```
from pyspark.sql.functions import col, count, desc
```

```
spark = SparkSession.builder.getOrCreate()
```

```
orders_df = spark.read.csv(  
    "/public/trendytech/retail_db/orders/*",  
    schema="order_id INT, order_date STRING, customer_id INT, order_status STRING"  
)
```

👉 Why DataFrame?

- Schema-aware
 - Optimized by Catalyst Optimizer
 - Uses Tungsten execution engine (faster than RDD)
-

◆ Use Case 1: Orders per Category

```
orders_df.groupBy("order_status") \
    .agg(count("*").alias("order_count")) \
    .orderBy(desc("order_count")) \
    .show()
```

💡 What changed vs RDD:

RDD	DataFrame
map → reduceByKey	groupBy + agg
manual parsing	schema-based
Python execution	JVM optimized

◆ Use Case 2: Premium Customers (Top 10)

```
orders_df.groupBy("customer_id") \
    .agg(count("*").alias("order_count")) \
    .orderBy(desc("order_count")) \
    .limit(10) \
    .show()
```

◆ Use Case 3: Distinct Customers

```
orders_df.select("customer_id").distinct().count()
```

◆ Use Case 4: Customers with Max CLOSED Orders

```
orders_df.filter(col("order_status") == "CLOSED") \
    .groupBy("customer_id") \
    .agg(count("*").alias("closed_orders")) \
    .orderBy(desc("closed_orders")) \
    .show()
```

🔥 Why DataFrames Win (Senior Insight)

Feature	Benefit
Catalyst Optimizer	Auto query optimization
Tungsten Engine	Memory + CPU efficient
Columnar format	Faster aggregation
Predicate pushdown	Reads less data

👉 In real companies, RDDs are rarely used unless needed

◆ 2. DAG (Directed Acyclic Graph) – How Spark Executes This

Let's break this down properly.

💡 What is DAG?

A DAG = Execution Plan

It shows how Spark will process your data step-by-step.

◆ Example (Orders per Category)

Code:

```
orders_df.groupBy("order_status").count().show()
```

◆ Logical DAG (High Level)

Read Data

↓

Project (select order_status)

↓

GroupBy (order_status)

↓

Aggregation (count)

↓

Sort (optional)

↓

Action (show)

◆ Physical DAG (What Actually Happens)

Stage 1 (No Shuffle)

- Read file
- Extract column

↓ SHUFFLE (key = order_status)

Stage 2

- Aggregate counts per key
- Final merge

↓

Driver (show result)

🔥 Key Concept: Stages

Spark splits DAG into stages based on shuffle:

Stage Type	Example
Narrow transformation	map, filter
Wide transformation	groupBy, reduceByKey

👉 Shuffle = stage boundary

◆ Visual Representation

[Input Data]

↓

(map / filter) ← Stage 1 (narrow)

↓

===== SHUFFLE =====

↓

(reduce / groupBy) ← Stage 2 (wide)

↓

(action)

◆ How to See DAG in Real Life

Option 1: Spark UI

When your job runs:

- Open Spark UI
- Go to Stages / Jobs tab

You'll see:

- DAG visualization
 - Shuffle details
 - Task execution
-

Option 2: Explain Plan

```
orders_df.groupBy("order_status").count().explain(True)
```

Output Types:

- Logical Plan
 - Optimized Plan
 - Physical Plan
-

🔥 Senior-Level Insight

⚠️ Performance depends on:

1. Number of shuffles
 2. Data skew
 3. Partition size
-

◆ Example Optimization Thought Process

Bad:

```
groupBy().count()
```

Better:

```
repartition("order_status").groupBy().count()
```

◆ Final Takeaway

RDD vs DataFrame

RDD	DataFrame
-----	-----------

Manual	Declarative
--------	-------------

No optimization	Auto-optimized
-----------------	----------------

Slower	Faster
--------	--------

Rarely used now	Industry standard
-----------------	-------------------

DAG Mental Model

Think like this:

Transformations = Plan

Action = Execute

Shuffle = Expensive boundary
